

Chapter 1

1. There are at least six domain-specific languages used in the little system I cobbled together to write and publish this book. What are they?

Markdown, Jinja2, Makefile, SASS, CSS, HTML

2. Get a “Hello, world!” program written and running in Java. Set up whatever makefiles or IDE projects you need to get it working. If you have a debugger, get comfortable with it and step through your program as it runs.

<https://github.com/vincent-24/CS4080/blob/main/HW1/Main.java>

3. Do the same thing for C. To get some practice with pointers, define a [doubly linked list](#) of heap-allocated strings. Write functions to insert, find, and delete items from it. Test them.

<https://github.com/vincent-24/CS4080/blob/main/HW1/main.c>

Chapter 2

1. Pick an open source implementation of a language you like. Download the source code and poke around in it. Try to find the code that implements the scanner and parser. Are they handwritten, or generated using tools like Lex and Yacc? (.l or .y files usually imply the latter.)

I chose Java as the open source implementation.

- a. The parser is located in

<https://github.com/openjdk/jdk/blob/master/src/jdk.compiler/share/classes/com/sun/tools/javac/parser/JavacParser.java> and it is handwritten.

- b. The scanner is located in

<https://github.com/openjdk/jdk/blob/master/src/jdk.compiler/share/classes/com/sun/tools/javac/parser/JavaTokenizer.java> and is also handwritten.

1. Just-in-time compilation tends to be the fastest way to implement dynamically typed languages, but not all of them use it. What reasons are there to *not* JIT?

Language implementations often avoid JIT compilation because it minimizes memory usage, reduces startup time, and allows for portability across different architectures.

- Most Lisp implementations that compile to C also contain an interpreter that lets them execute Lisp code on the fly as well. Why?

The interpreter is required to execute macros during compilation, reducing overhead time.

Chapter 3

- Write some sample Lox programs and run them (you can use the implementations of Lox in [my repository](#)). Try to come up with edge case behavior I didn't specify here. Does it do what you expect? Why or why not?

```

≡ main.lox
1  var a = 8;
2  var b = 4;
3  var c = 0;
4
5  print a / b;
6  print a / c;

craftinginterpreters % ./clox main.lox

2
inf

craftinginterpreters % ./jlox main.lox

2
Infinity

```

This is not the output I expected since the program should output a runtime error for dividing by zero.

- This informal introduction leaves a *lot* unspecified. List several open questions you have about the language's syntax and semantics. What do you think the answers should be?

How do we add variables into print statements such as

**printf("Variable: %d", variable); in C and
system.out.println("Variable: " + variable); in Java?**

**I think it should be something similar to the formatted syntax in C and
should be**

print "Variable: %var", variable;

3. Lox is a pretty tiny language. What features do you think it is missing that would make it annoying to use for real programs? (Aside from the standard library, of course.)

Error handling would make it really difficult to work with since certain errors such as dividing by zero do not throw runtime errors.